COURSE BRIEF / ROS 2 JAZZY / 1 NOVEMBER 2026

Nodes to Maps

A practical introduction to ROS 2, from the first message to a phone-sensor mapping project.

This course is both a five-lecture crash course and a reusable self-learning guide. Your robotics workbench (00) prepares Ubuntu and Jazzy. From a robot to the ROS graph (01) connects sensing, processing and actuation to ROS vocabulary, an upstream source build and live graph inspection. Later lectures build toward a phone sensor pipeline.

Platform	Teaching format	Capstone
ROS 2 Jazzy Ubuntu 24.04 LTS	Five lectures 9 guided hours, plus setup	Android phone + laptop Optional monocular SLAM

What learners will be able to do

- Explain the robot feedback loop, why ROS is middleware rather than an operating system, and how to inspect a live graph.
- Build Python and C++ publishers/subscribers and use generated custom interfaces.
- Choose between topics, services and actions; configure nodes with parameters and launch files.
- Explain QoS compatibility, coordinate frames, transforms and timestamp-dependent data.
- Receive phone camera and motion data in ROS, inspect it, and record/replay a rosbag.
- Describe calibration and demonstrate sparse monocular mapping after the backend and phone have been qualified.

Audience and preparation

No robotics background is assumed. Learners need basic Python functions and classes; C++ scaffolds support the third lecture. Setup takes about 1-2 hours, with a separate instructor rehearsal for the phone lab. A laptop with 16 GB RAM and 30 GB free disk is comfortable. Use an Android 8.0+ phone. The same native app connects by wired USB or wireless over the phone hotspot.

Timing that matches the current course

Lectures 1-4 take 90 minutes each. The full fifth lecture takes 180 minutes, for nine contact hours. For five 90-minute meetings, teach phone transport and a prepared mapping demonstration in the final meeting, then assign the full mapping run as a follow-up lab. Setup is additional.

The instructor uses Windows 11 with Ubuntu 24.04 in WSL 2. Native Ubuntu follows the same ROS commands. Other platform options are introduced with their support limits. Real phone and live mapping qualification remains a separate step.

LECTURE PLANS / 01-02

Understand the graph. Then build it.

Each session combines an explanation, a working example, a deliberate fault and evidence that the learner can interpret.

LECTURE 01 / 90 MIN

From a robot to the ROS graph

Outcome: Explain the robot loop and vocabulary; build and inspect a small upstream publisher.

Session sequence	Minutes
Robot feedback loop and computer stack	15
ROS framework and core vocabulary	25
DDS and delivery contracts	10
Git source, colcon build and publisher run	20
Turtle interfaces, diagnosis and checkpoint	20

Practical work: Trace the robot loop, clone/pull the official Jazzy example, build one publisher, distinguish package/executable/node names and inspect its remapped topic. Use the same concepts to diagnose turtlesim.

Checkpoint: Explain the robot loop, choose a communication pattern and diagnose a graph using observed evidence. Platform setup, Linux practice and initial turtle startup are in lecture 00.

LECTURE 02 / 90 MIN

Write your first Python nodes

Outcome: Build a publisher, subscriber and generated interface.

Session sequence	Minutes
Supplied package build and workspace	10
Complete Python publisher	20
Packaging and executable entry points	15
Subscriber and custom interfaces	20
Hands-on implementation lab	25

Practical work: Create rclpy nodes with timers and callbacks, register an installed executable, build/source the workspace, and exchange a meaningful custom message.

Checkpoint: Run the installed nodes and explain the difference between source code, a Node object, a package and an executable. Explain when callbacks run and how interfaces are generated.

LECTURE PLANS / 03-04

Make the system communicate reliably.

The same ROS graph supports multiple languages and interaction styles. Learners connect those ideas to timing and geometry.

LECTURE 03 / 90 MIN

The same graph, in C++

Outcome: Connect C++ to Python and reason about QoS.

Session sequence	Minutes
Translate familiar node concepts into C++	15
C++ publisher/subscriber and CMake build	25
Quality of Service compatibility	20
Break delivery, inspect and repair	20
Checkpoint	10

Practical work: Build the supplied rclcpp examples, connect them to Python peers and compare reliability settings. Inspect offered/requested QoS when a typed topic is visible but messages fail to arrive.

Checkpoint: Demonstrate cross-language delivery and justify a compatible QoS choice using endpoint evidence. The session provides a C++ scaffold rather than a full language course.

LECTURE 04 / 90 MIN

Connect a complete system

Outcome: Choose interactions and transform a measurement between frames.

Session sequence	Minutes
Service request/response	15
Action goals, feedback, results and cancellation	15
Parameters and launch files	20
TF frames, point transforms and time	25
Checkpoint	15

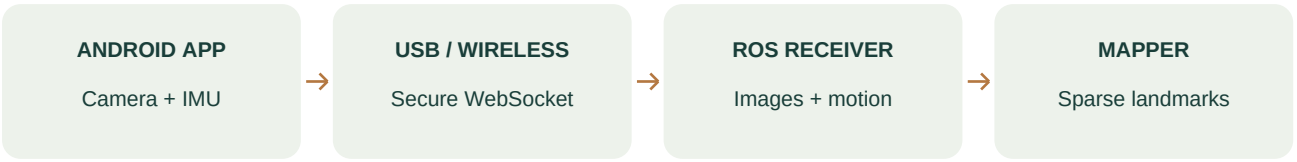
Practical work: Call an area service, run and cancel a counting action, configure and launch nodes, and work through a rotated/translated point. Relate map, odom, base and optical frames to lookup direction and time.

Checkpoint: Choose a topic, service or action for a task; explain a source-to-target transform and why a phone camera-to-IMU transform requires measured geometry.

LECTURE PLAN / 05

A phone that helps build a map

A 180-minute guided capstone brings sensing, networking, ROS inspection, geometry and recording into one reproducible experiment.



The phone app sends camera frames and independent accelerometer/gyroscope samples to the laptop. The ROS bridge validates and timestamps them, then publishes images, camera information, motion and diagnostics. A separate ORB-SLAM3 adapter consumes calibrated images.

Session sequence	Minutes
Pipeline and clocks	25
Pair the Android app; inspect ROS topics	30
Frames and calibration review	20
Break	10
Mapping and rosbag recording	50
Diagnose faults using evidence	25
Discussion, report and extensions	20

What happens in the lab

- Pair using a reachable secure WebSocket endpoint, session token and server certificate fingerprint. Choose Option 1: wired USB with ADB reverse, or Option 2: wireless over the phone hotspot. WSL forwarding is documented for both.
- Display the camera stream, inspect rates and native motion components, and identify the phone camera timestamp source.
- Load calibration measured for the transmitted camera mode. Initialize mapping with slow translation through a textured scene, then record and replay a short consented run.

What students submit

Phone/app identity, camera mode and clock source; calibration and backend versions; commands and rate evidence; a bag manifest/checksum; one diagnosed failure; and a real map screenshot/trajectory when mapping succeeds. Rubric: transport 25%, timing/frames/calibration 30%, reproducibility 25%, diagnosis 20%.

Scope and extension path

The baseline produces sparse monocular landmarks with arbitrary scale. IMU streams support inspection; the supplied mapper uses images. Camera-IMU fusion needs measured extrinsics, timing/noise qualification and an integration stage. Dense metric reconstruction is a further project. Prepare TLS, backend compilation, calibration and a reference recording before teaching this session.

THE WEBSITE / LEARNING EXPERIENCE

One guide, two ways to learn

The static website uses the same authoritative Markdown notes as the standalone reading bundle.

Self-study mode	Lecture mode
Full reading guide, exercises, quiz feedback and model explanations. Students can revisit sections and follow linked references.	Section-by-section presentation with navigation controls and a timer. Instructors use the same examples and checkpoints during class.

Tools supplied to every learner

- Search across course topics; navigate lessons, resource pages and section outlines.
- Use interactive concept labs for the ROS graph, communication behavior, transforms and phone bandwidth.
- Answer 35 scenario-based quiz questions with explanatory feedback, plus 13 written prompts with model explanations and evidence criteria.
- Save section progress, personal notes and written responses locally in the browser; export progress/notes from Resources.
- Copy commands from dark panels; compare results with blue EXPECTED OUTPUT panels that have no Copy button. Inspect source files and use printable reading views.
- Use purposeful original concept diagrams, an attributed upstream RViz screenshot, and labelled AI-generated Windows launch and equipment illustrations. Small terminal results appear as text.

Development and practical reference guides

Guide	What it covers
A / Android app	Camera2, SensorManager, timestamps, secure transport and builds.
B / ROS receiver	Packet validation, image decoding, clocks and ROS publication.
C / Calibration	Camera calibration, axes, camera-IMU geometry and qualification.
D / ROS workflows	Services, actions, bags, TF and further projects.
E / USB or wireless	Android ADB reverse or Wi-Fi over the phone hotspot.
F / WSL capstone	Forwarding and graphics for the complete phone project.
WSL foundations	Three focused appendices: Windows/Linux files, USB sensor attachment, and networking/localhost.

Supporting resources

Resources also include a CLI field guide, frames/transforms reference, phone-lab preparation, transport contracts, visual guide, troubleshooting, references/glossary, course scope and learning outcomes, and worked solutions/rubric.

LEARNING MATERIALS / GETTING STARTED

Your course materials

Use the website during lessons and the downloadable notes and code for independent practice.

Provided material	Purpose
Static website ZIP	Host the full guide with illustrations, learning modes and linked downloads.
Illustrated Markdown notes ZIP	Read lessons, annexes and assessments offline, with images and linked artifacts.
Android test APK + checksum	Development-signed classroom sensor client for Android 8.0+.
Android Studio source	Inspect or modify the native app and follow its build instructions.
ROS workspace ZIP	Four beginner packages: interfaces, Python, C++ and bringup.
Optional SLAM adapter ZIP	Monocular ROS adapter, calibration converter and preparation guides; external engine required.

Prepare your learning environment

Start with Ubuntu 24.04 and ROS 2 Jazzy. Follow lecture 00 to prepare your terminal and run turtlesim. In lecture 01, clone and build the upstream publisher example. Build the supplied course workspace in lecture 02.

For the capstone, connect the phone by USB or wireless, inspect its camera and motion topics, and calibrate the exact camera mode. The optional mapper needs a separately prepared ORB-SLAM3 engine and vocabulary. Follow the phone lab guide for recording and replay.

Choose how to study

Open the course in self-study mode for complete explanations and exercises, or use lecture mode to follow a session. Download the illustrated Markdown notes for offline reading. Save your progress and notes in the browser, and export them from Resources.

Further reading

Continue learning with the official references linked throughout the course:

ROS 2 Jazzy documentation

Android Camera2

WSL networking

ORB-SLAM3 project

Original course text/diagrams: CC BY 4.0. RViz reference screenshot: Clear BSD, with upstream attribution/license retained. Generated illustrations are identified in the asset record. Core/phone-client code: Apache-2.0. Optional adapter: GPL-3.0-or-later.